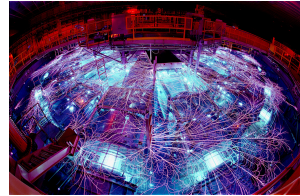
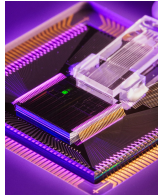


*Exceptional service in the national interest*



# Formal Verification of High Consequence Systems

PDEs: Verification, Mathematics, and Algorithms in the Age of AI

Samuel D. Pollard

17 Sep 2026



Introduction

Q Framework

Foundational Verification in Rocq

Probabilistic Programming for Hardware Robustness Analysis

Elaenia: Floating-Point Error Analysis



Introduction

Q Framework

Foundational Verification in Rocq

Probabilistic Programming for Hardware Robustness Analysis

Elaenia: Floating-Point Error Analysis



High consequence systems are systems where a single failure can have catastrophic consequences. Sandia National Laboratories develops high consequence systems and uses a variety of verification and validation (V&V) techniques to ensure safe, robust, and secure operation of these systems. This talk focuses on one aspect of V&V called formal methods and gives an overview of its use at Sandia. We discuss two large verification efforts at the labs: Q Framework and foundational verification in Rocq. Q Framework provides model based systems engineering verification using model checking, refinement proofs, and automated reasoning. Foundational verification includes formal proofs of C programs using the Verified Software Toolchain and CompCert. We conclude with a discussion of research projects at Sandia including automated reasoning for floating point error analysis and probabilistic programming languages for hardware fault tolerance.



# Overview of Sandia



- Sandia National Labs is funded by the US Federal Government and part of the National Nuclear Security Administration (NNSA), itself part of the Department of Energy
- Sandia has many research areas centered around national security
- One is assurance of high-consequence embedded control systems



Livermore, California site

# High Consequence and Safety Critical Are on a Spectrum



safety critical

high consequence



# High Consequence and Safety Critical Are on a Spectrum



safety critical

high consequence



# High Consequence and Safety Critical Are on a Spectrum



safety critical

high consequence

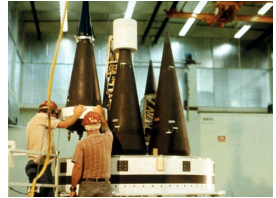


# High Consequence and Safety Critical Are on a Spectrum



safety critical

high consequence



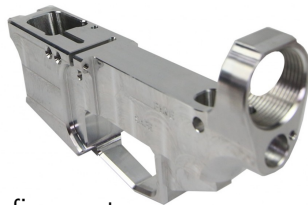
# Proving Theorems About Computer Programs



- Fundamentally, computer science is the study of abstractions
- Theorems in the abstract system should hold in refined system
- We use meaning-preserving translations, formal proof, trace inclusion, simulation, Hoare logic



Abstraction



Refinement



Introduction

Q Framework

Foundational Verification in Rocq

Probabilistic Programming for Hardware Robustness Analysis

Elaenia: Floating-Point Error Analysis

## Q Framework Enables MBSE Verification



- MBSE = Model-Based Systems Engineering
- Move from documents to models as fundamental units



## Q Framework Enables MBSE Verification



- MBSE = Model-Based Systems Engineering
- Move from documents to models as fundamental units
- PL perspective: enrich models with semantics
- FM perspective: enrich models with *formal* semantics

## Q Framework Enables MBSE Verification



- MBSE = Model-Based Systems Engineering
- Move from documents to models as fundamental units
- PL perspective: enrich models with semantics
- FM perspective: enrich models with *formal* semantics
- Existing tools (Event-B [1]) provide refinement proofs
- We need refinement between models, and models into C

# Q Framework Enables MBSE Verification

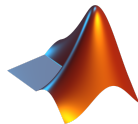
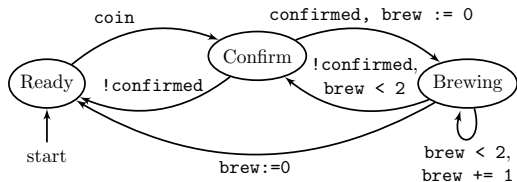


- MBSE = Model-Based Systems Engineering
- Move from documents to models as fundamental units
- PL perspective: enrich models with semantics
- FM perspective: enrich models with *formal* semantics
- Existing tools (Event-B [1]) provide refinement proofs
- We need refinement between models, and models into C
- Q Framework provides (mostly) automated refinement proofs

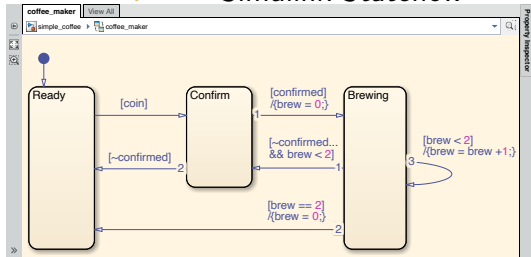


Imminently open source! <https://github.com/sandialabs/qq/>

# What is a Model?



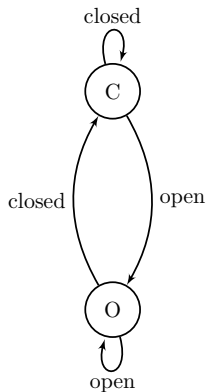
Simulink Stateflow



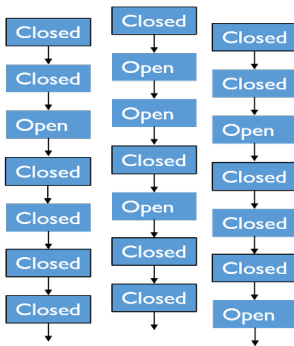
# Refinement Preserves Safety and LTL Properties



- LTL = Linear Temporal Logic
- Refinement selects a new machine that subsets the traces of the old machine



Refinement set of traces



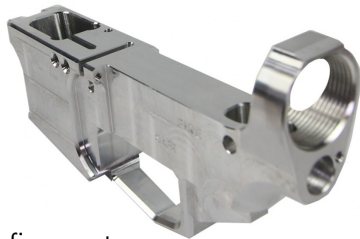
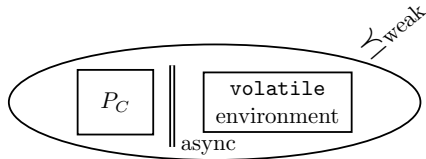
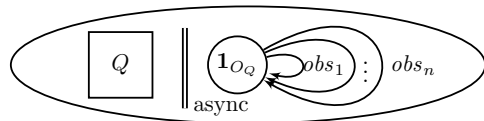
Abstraction set of traces



# Our Refinement Argument in Q Framework

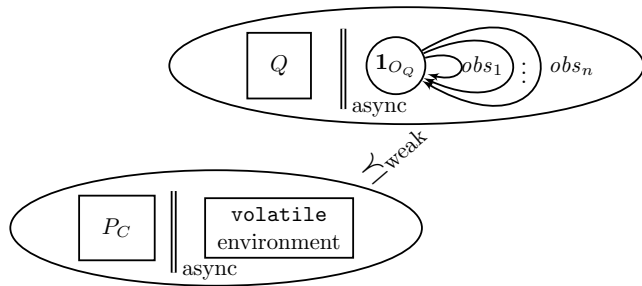


Abstraction



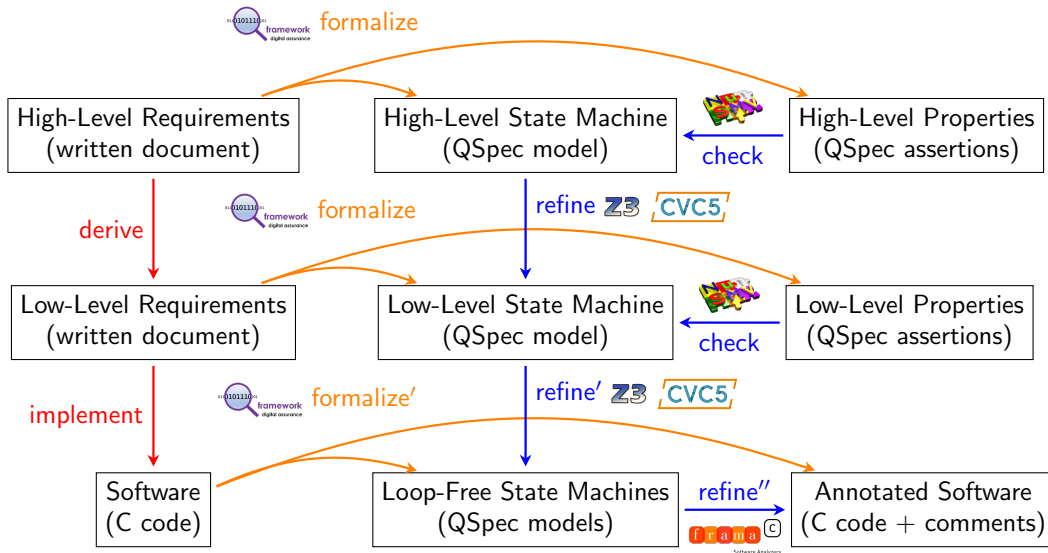
Refinement

# Compositional Software Requires Weak Refinement



- Strict refinement requires moment-by-moment subsumption of behavior
- Parallel asynchronous composition  $||_{\text{async}}$  must permit *stutter steps*
- Corollary: unbounded stutter steps equates to nontermination

# Q Framework Architecture





# Hoare Logic Allows Reasoning About Imperative Programs



Hoare logic has as its base object a “Hoare triple”

$$\{P\}c\{Q\}$$

- $c$  is a program fragment
- $P$  is a precondition
- $Q$  is a postcondition

$$\{y = 2\}$$

$$x := y + 1$$

$$\{y = 2 \wedge x = 3\}$$

# Frama-C Verifies Programs Using Hoare Logic Written in ACSL



- ACSL = ANSI C Specification Language
- A first-order logic about C programs



Software Analyzers

```
/*@  
  requires \valid(a) ∧ \valid(b);  
  ensures *a == \old(*b)  
         ∧ *b == \old(*a);  
  assigns *a, *b;  
*/  
void swap(int *a, int *b)  
{  
  int temp = *a;  
  *a = *b;  
  *b = temp;  
  return;  
}
```



Introduction

Q Framework

Foundational Verification in Rocq

Probabilistic Programming for Hardware Robustness Analysis

Elaenia: Floating-Point Error Analysis

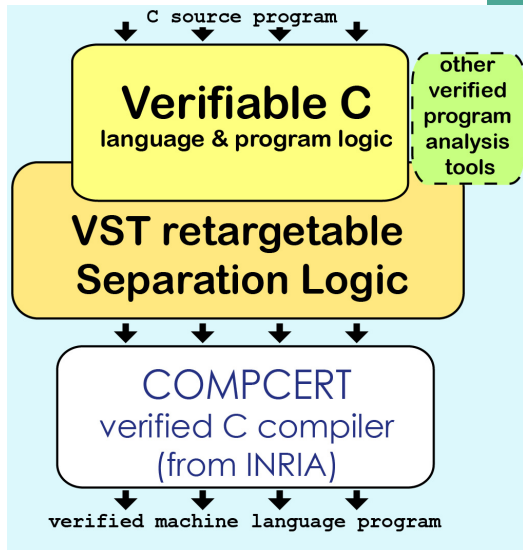
# Some Theorems Require Higher Order Logics



- Real programs contain higher-order features
  - function pointers
  - objects
- To reason about these compositionally, we need higher-order specifications.
- *security properties* are often of the form “an arbitrary adversary program can’t gain an advantage (that is, do better than random guessing) parametrically in some secret”



- Higher-order program logic
- More expressive than Frama-C's logic, but less automated.
- Built upon CompCert C semantics
- Correctness proved from Rocq specification down to machine assembly
  - A formally verified linker “coming soon”





CompCert proves the observable behavior of a translated program  $tp$  improves upon the observable behaviors of  $p$  [2, 3].

**Theorem** `transf_c_program_correct:`

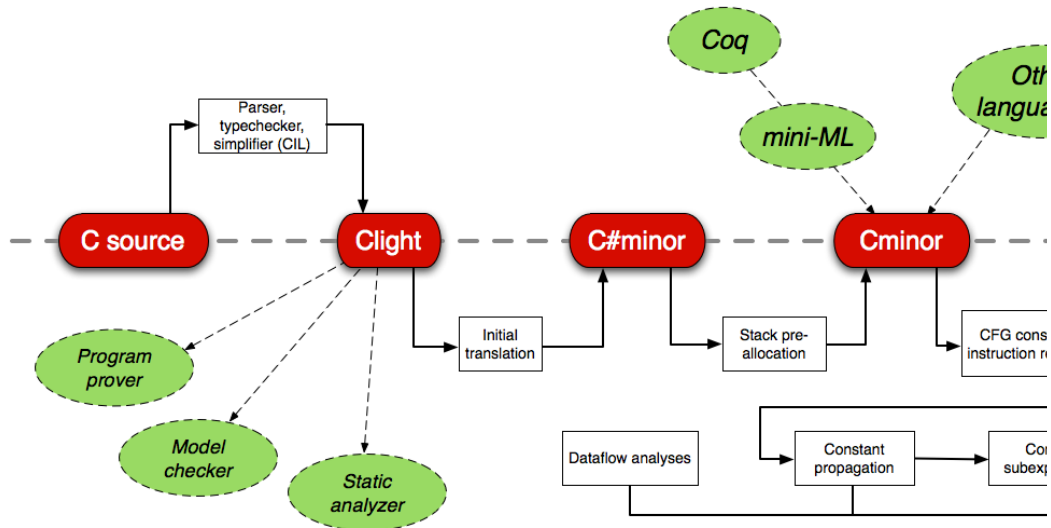
```
forall p tp,  
transf_c_program p = OK tp  $\rightarrow$   
backward_simulation (Csem.semantics p) (Asm.semantics tp).
```

**Proof.**

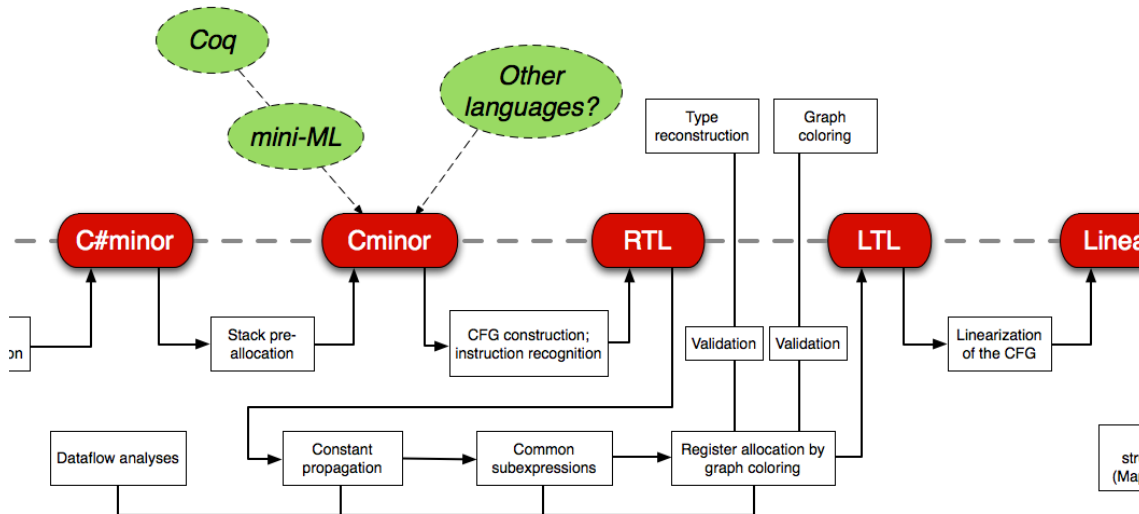
```
intros. apply c_semantic_preservation.  
apply transf_c_program_match; auto.
```

**Qed.**

# CompCert Does This Through Intermediate Languages, Pt. 1

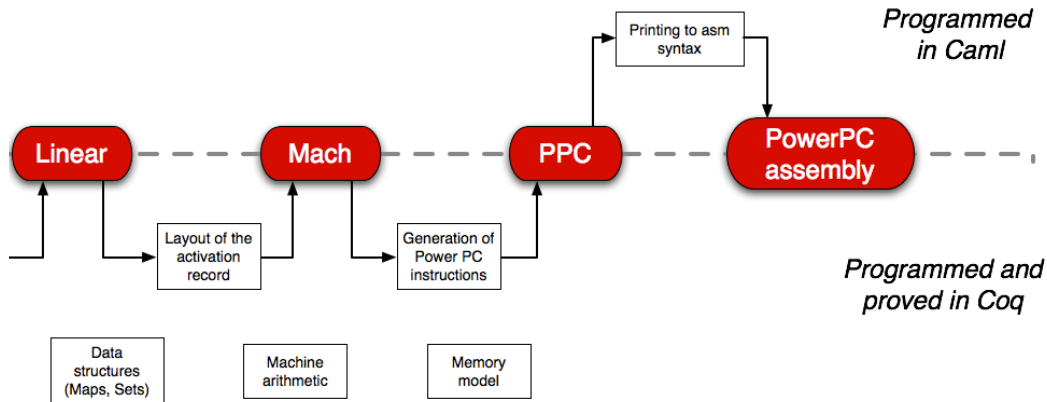


## CompCert Does This Through Intermediate Languages, Pt. 2





## CompCert Does This Through Intermediate Languages, Pt. 3





Introduction

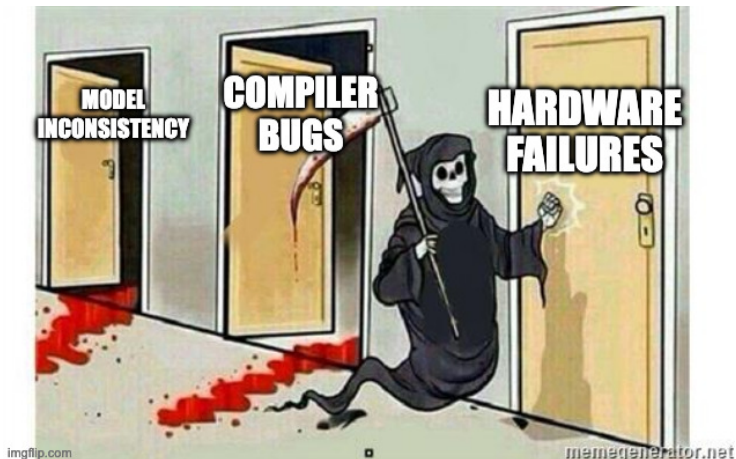
Q Framework

Foundational Verification in Rocq

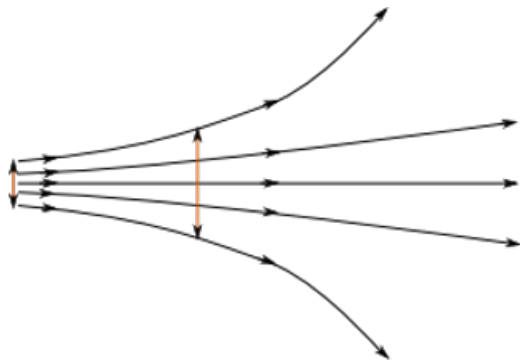
Probabilistic Programming for Hardware Robustness Analysis

Elaenia: Floating-Point Error Analysis

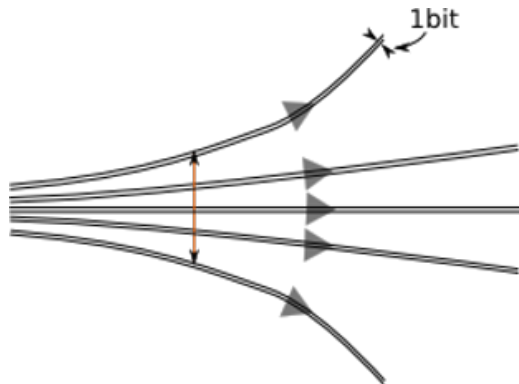
# What If a Computer Stops Acting Like a Digital System?



# Ionizing Radition Can Break the Digital Abstraction

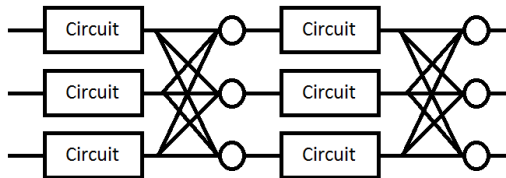
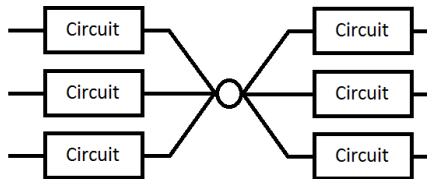
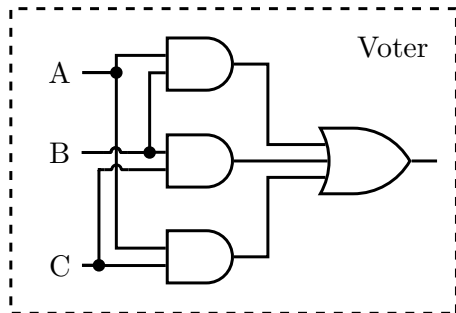


Chaotic dynamical system: divergent trajectories



Generic digital system: convergent within each “channel” (digital path), divergent between “channels”

# We Want to Evaluate the Design Space of Redundancy

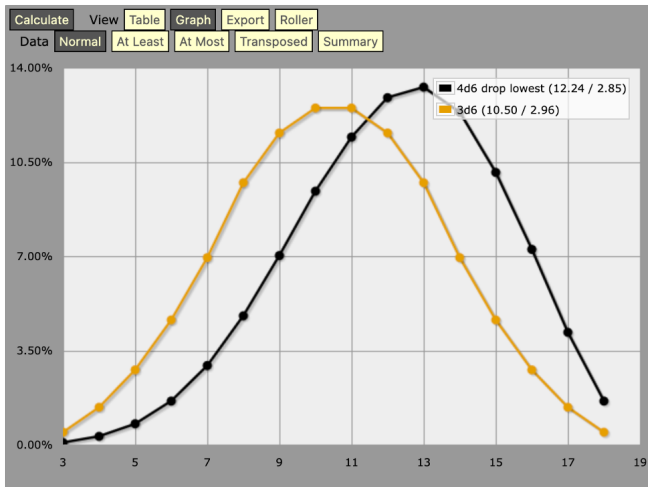


CC-BY-SA [6]

# A Probabilistic Program Generates a Probability Distribution



```
output [highest 3 of 4d6]
  named "5e"
output 3d6
  named "down the line"
```

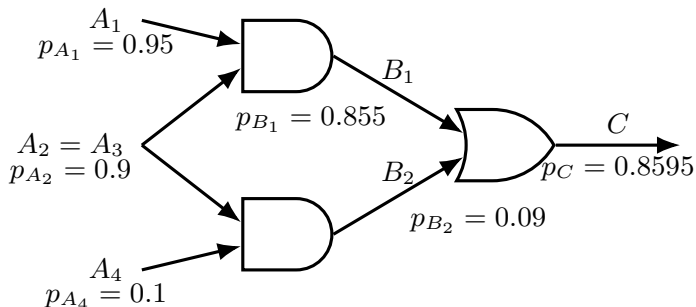


<https://anydice.com/program/ff1>

## Given Failure Probabilities of Each Gate, We Build a Probability Distribution



- Suppose  $A_1 \dots A_4$ ,  
are wires from other  
components



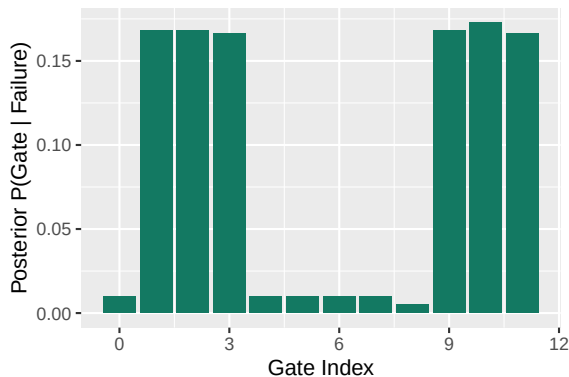
# Circulette Translates Verilog into a Probability Distribution



```
module mux2(select, d, q);  
  input[1:0] select;  
  input[3:0] d;  
  output q;  
  reg q;  
  wire[1:0] select;  
  wire[3:0] d;  
  always @(d or select)  
    q = d[select];  
endmodule
```

Circulette  
(+ Roulette + WMC)

Failure rates for mux2



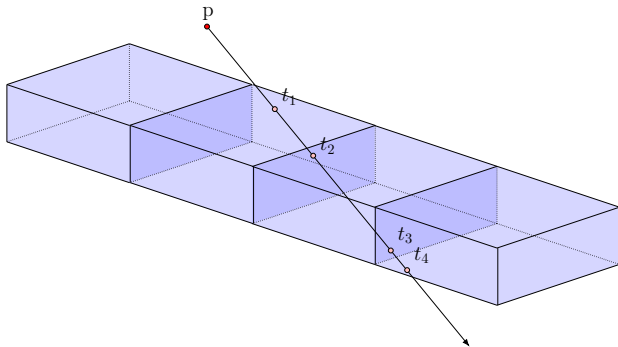
Now open source! <https://github.com/sandialabs/circulette/>



# Circulette Can Integrate Realistic Physical Modeling



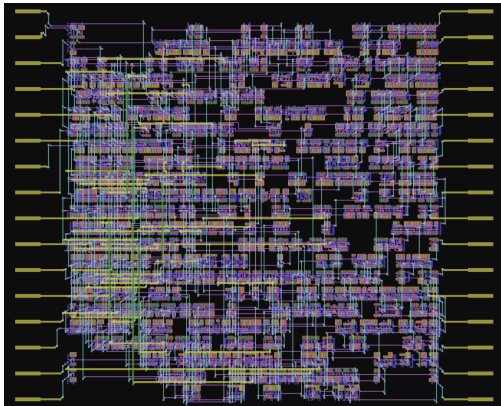
- Currently: model a constant failure rate based total ionizing dose
- Future work: account for substrate, layout, type of radiation, and dependencies



# Circulette Can Integrate Realistic Physical Modeling



- Currently: model a constant failure rate based total ionizing dose
- Future work: account for substrate, layout, type of radiation, and dependencies

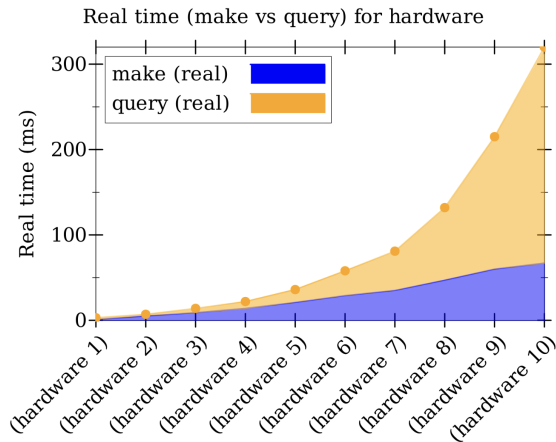


[https://znah.net/tiny\\_explorer/viewer.html?gds=gallery%2Fmultiplier8.oas](https://znah.net/tiny_explorer/viewer.html?gds=gallery%2Fmultiplier8.oas)

## Next Question: Will PPLs Run Into Scalability Issues?



- Exact inference is  $\#P$ -Hard
- Hypothesis: heuristics and domain-specific knowledge can obviate this computational problem à la SMT



Source: [5]



Introduction

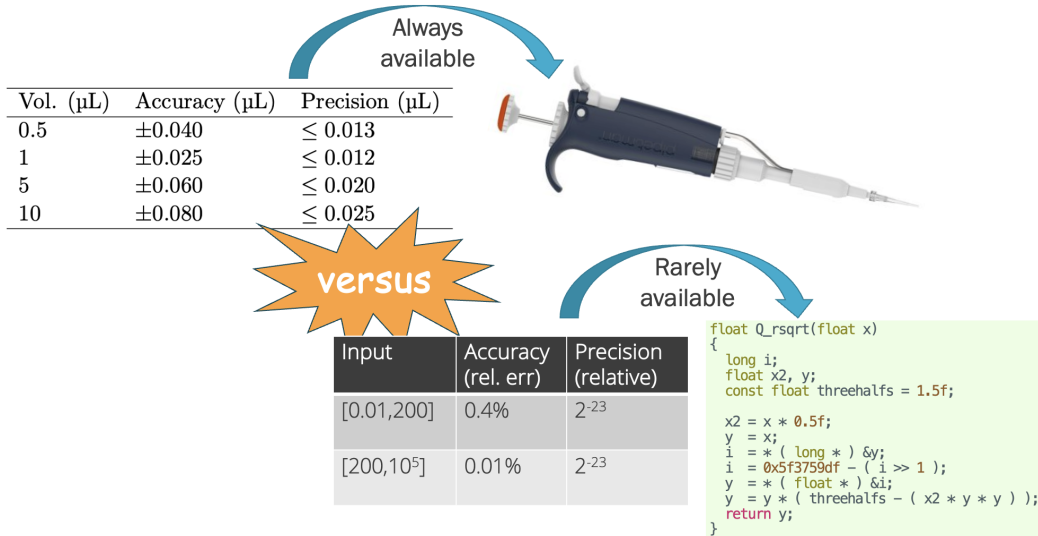
Q Framework

Foundational Verification in Rocq

Probabilistic Programming for Hardware Robustness Analysis

Elaenia: Floating-Point Error Analysis

# Software Is Not Treated Like Other Scientific Instruments



# Key Hypothesis: the C Code *Is* the Spec



- We must meet the developers where they are, except...
- C is an impoverished language for math
- C + ACSL is not

```
/*@ requires 0.0 <= x <= 1.0; */  
// |real - floating-point| <= 6.834e-16  
double sqrootp1(double x)  
{  
    return 1.0 + 0.5 * x  
        - 0.125 * x * x  
        + 0.0625 * x * x * x  
        - 0.0390625 * x * x * x * x;  
}
```

# Key Hypothesis: the C Code *Is* the Spec



precondition

- We must meet the developers where they are, except...
- C is an impoverished language for math
- C + ACSL is not

```
/*@ requires 0.0 <= x <= 1.0; */  
// |real - floating-point| <= 6.834e-16  
double sqrootp1(double x)  
{  
    return 1.0 + 0.5 * x  
        - 0.125 * x * x  
        + 0.0625 * x * x * x  
        - 0.0390625 * x * x * x * x;  
}
```

# Key Hypothesis: the C Code *Is* the Spec

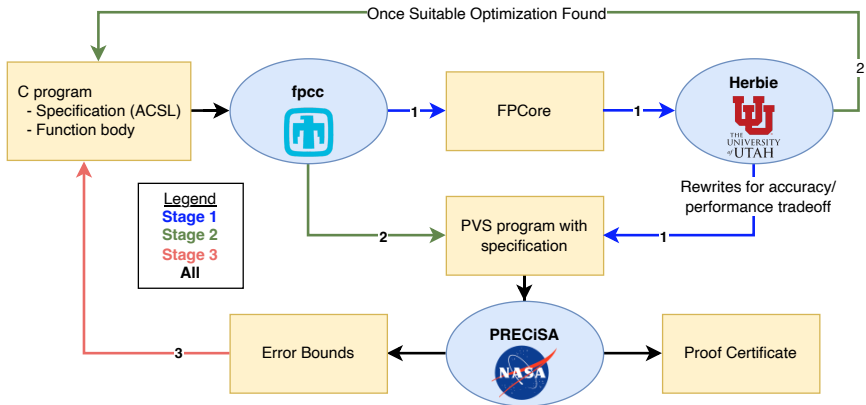


- We must meet the developers where they are, except...
- C is an impoverished language for math
- C + ACSL is not

```
/*@ requires 0.0 <= x <= 1.0; */ postcondition  
// |real - floating-point| <= 6.834e-16  
double sqrootp1(double x)  
{  
    return 1.0 + 0.5 * x  
        - 0.125 * x * x  
        + 0.0625 * x * x * x  
        - 0.0390625 * x * x * x * x;  
}
```



# Elaenia Enables Analysis of Many Floating-Point Implementations



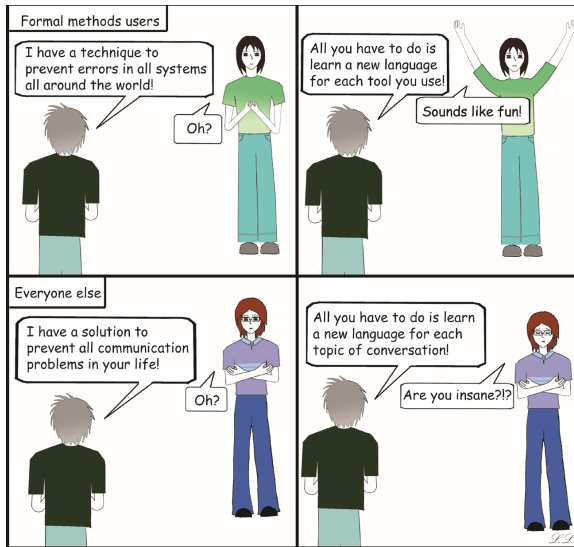
Now open source! <https://github.com/sandialabs/elaenia/>



# Conclusion

- This is the product of a team of ~20 staff + interns + university collaborators
- Special thanks to Northeastern University, University of Utah

<https://proof.sandia.gov>

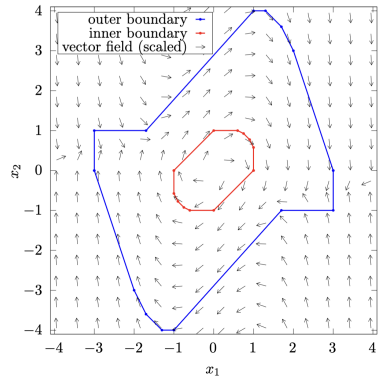




- [1] ABRIAL, J.-R., BUTLER, M., HALLERSTED, S., HOANG, T. S., MEHTA, F., AND VOISIN, L. Rodin: an open toolset for modelling and reasoning in Event-B. *International Journal on Software Tools for Technology Transfer* 12, 6 (2010), 447–466.
- [2] ABSINT ANGEWANDTE INFORMATIK GMBH. CompCert, 2026.  
<https://github.com/AbsInt/CompCert/blob/master/driver/Compiler.v#L446>.
- [3] LEROY, X. The compcert c verified compiler: Documentation and user's manual, version 3.18. Tech. rep., Collège de France and Inria, Aug. 2026. Available: <https://compcert.org/man/index.html>.
- [4] LIVESAY, M. R., AND RAWLINGS, B. C. Smt formulations of sufficient conditions for forward invariance in continuous dynamical systems. In *Proceedings of the 28th ACM International Conference on Hybrid Systems: Computation and Control* (New York, NY, USA, 2025), HSCC '25, Association for Computing Machinery.
- [5] TEJA, S. Roulette benchmarking results summary, Sept. 2026. Available: <https://smaran-teja.github.io/roulette-bench-data/scaling.html>.
- [6] TRIPLE MODULAR REDUNDANCY. Triple modular redundancy — wikipedia, the free encyclopedia, 2026.  
[https://en.wikipedia.org/wiki/Triple\\_modular\\_redundancy](https://en.wikipedia.org/wiki/Triple_modular_redundancy).



- Formal verification can be thought of as uncertainty quantification for ODEs/PDEs which are highly nonlinear and abstracted as discrete combinatorial switches
- Additionally, analog verification of electronic circuits using SMT [4]; and analysis using SPICE simulations with Xyce, Charon, NuGET





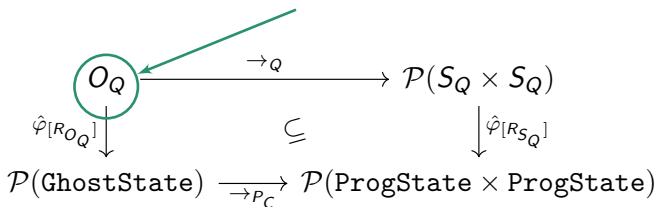
$$\begin{array}{ccc} O_Q & \xrightarrow{\rightarrow_Q} & \mathcal{P}(S_Q \times S_Q) \\ \hat{\varphi}[R_{O_Q}] \downarrow & \subseteq & \downarrow \hat{\varphi}[R_{S_Q}] \\ \mathcal{P}(\text{GhostState}) & \xrightarrow{\rightarrow_{P_C}} & \mathcal{P}(\text{ProgState} \times \text{ProgState}) \end{array}$$

- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $\mathcal{P}(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics

# Weak Simulation in Q Framework

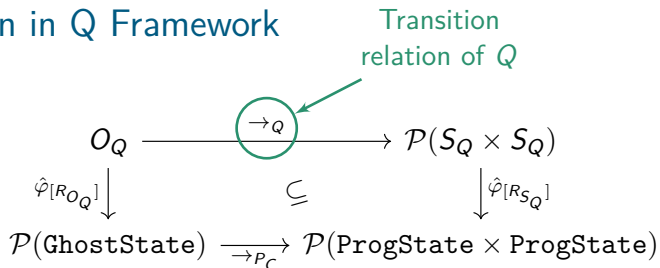


Observables in the LTS  $Q$



- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $\mathcal{P}(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics

# Weak Simulation in Q Framework

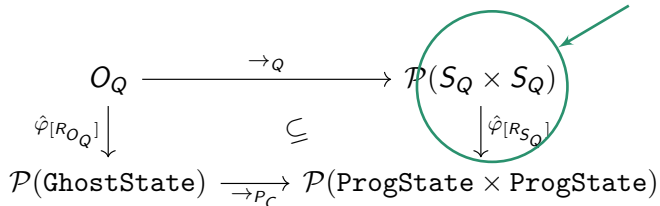


- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $\mathcal{P}(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics

# Weak Simulation in Q Framework



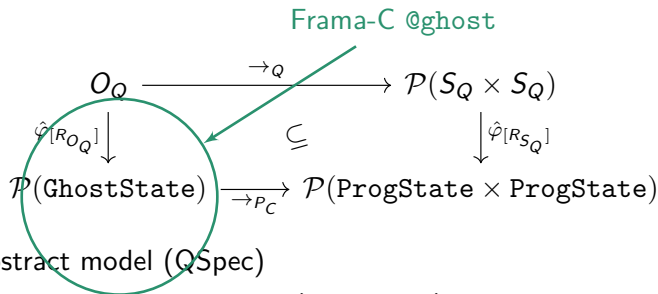
Relations over  
states in  $Q$



- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $P(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics

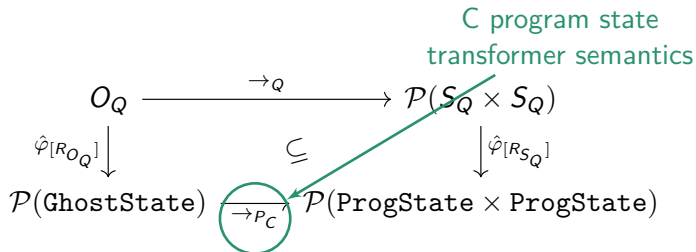


# Weak Simulation in Q Framework



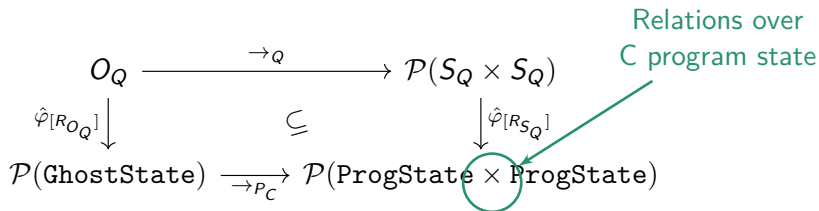
- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $\mathcal{P}(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics

# Weak Simulation in Q Framework



- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $\mathcal{P}(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics

# Weak Simulation in Q Framework



- $Q$  is the abstract model (QSpec)
- $P_C$  is the concrete implementation (C program)
- $\hat{\varphi}$  relates QSpec variables to predicates over C variables.
- $\rightarrow_Q$  is a Galois connection between  $O_Q$  and  $\mathcal{P}(S_Q \times S_Q)$
- This demonstrates a proof of weak simulation, provided we can think of  $P_C$  as a transition system: this is not trivial when considering C semantics